

MEDIATEK

MT8668 Hypervisor Multi Display Proxy- Wayland User Manual

版本: 1.0
出版日期: 2026-01-28

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2026-01-28	Mengxu Li	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 4

1 Proxy-Wayland 5

1.1 概述 5

1.1.1 Proxy-Wayland 的实现需求 5

1.1.2 Proxy-Wayland 的实现原理 5

1.1.3 Proxy-Wayland 的代码结构分布 5

1.2 缩略词 6

1.3 架构/进程概述 7

1.3.1 Hypervisor Proxy-Wayland 系统框架 7

1.3.2 Proxy-Wayland 模块结构分布 7

1.4 测试与分析 8

1.4.1 Proxy-Wayland 功能性测试与分析 8

1.4.2 Proxy-Wayland 稳定性测试与分析 9

附件一 附加条款 16

图片目录

图 1-1. Hypervisor Proxy-Wayland 系统框架 7

图 1-2. Proxy-Wayland 模块结构分布 8

图 1-3. Proxy-Wayland two-path display fps 10

图 1-4. Weston reboot latency 11

图 1-5. Proxy service reboot SIGTERM latency 12

图 1-6. Proxy service reboot SIGINT latency 13

图 1-7. Android reboot latency 14

表格目录

表 1-1. 缩略词.....6

1 Proxy-Wayland

1.1 概述

本章节主要介绍 MT8668 Hypervisor Proxy-Wayland 的实现需求、实现原理及代码结构分布。

1.1.1 Proxy-Wayland 的实现需求

当前，车载多媒体娱乐系统正迅猛发展，其背后的多域显示系统管理极为复杂。例如，中控屏、副驾屏、后枕屏、仪表盘等各个领域均需各自的显示屏。基于单一芯片，多屏的显示来自于不同的 OS，充分发挥不同 OS 的优势，如 Android 多媒体处理的高效性；Linux 的及时性和安全性。

然而，要实现单一芯片的多域功能，就不可避免涉及到虚拟技术。目前，车载行业较为流行的多域是 Linux + Android，其中一路 Linux 是 Host，其它路是 Guest。Proxy-Wayland 正是基于这种跨域需求，实现从 Android guest 将图形 buffer 送到 Linux host，透过 Wayland 进行显示，实现跨域显示的功能。

1.1.2 Proxy-Wayland 的实现原理

- Proxy-Wayland 的通信框架以 RPC binder 为基础。
- RPC binder 以 Vsock 为基础实现跨域。
- Android 端 HWC 通过 Proxy-Wayland binder client 接口将 frame 透过基于 Vsock 的 RPC binder 实现跨域。
- Linux 端 Proxy-Wayland binder service 接收到 frame 后，调用 wayland-client API 实现显示功能。

1.1.3 Proxy-Wayland 的代码结构分布

- **Android**
 - Code path: vendor/mediatek/proprietary/external/MultiVirtualDisplay
 - Android-client
 - Proxy-Wayland HWC 接口类
 - Proxy-Wayland display client 管理类
 - Proxy-Wayland display client 类
 - libvirtual-display
 - Binder Bp & Bn 类
 - libdmabufhelper
 - DMA buffer 传递的参数转换接口类

- Linux
 - Code path: src/multimedia/multi-virtual-display
 - yocto-service/virtual-display-service
 - Binder service main 函数
 - yocto-service/libvirtual-display-service
 - Binder service 实现类
 - Display client 管理类
 - Display client 实现类
 - libvirtual-display
 - Binder Bp & Bn 类
 - libdmabufhelper
 - DMA buffer 传递的参数转换接口类
 - BB file path
 - meta/meta-mediatek-mt8668/recipes-multimedia/hyper-virtual-display-service
 - meta/meta-mediatek-mt8668/recipes-multimedia/hyper-libvirtual-display-service
 - meta/meta-mediatek-mt8668/recipes-multimedia/hyper-libvirtual-display
 - meta/meta-mediatek-mt8668/recipes-multimedia/hyper-libdmabufhelper

1.2 缩略词

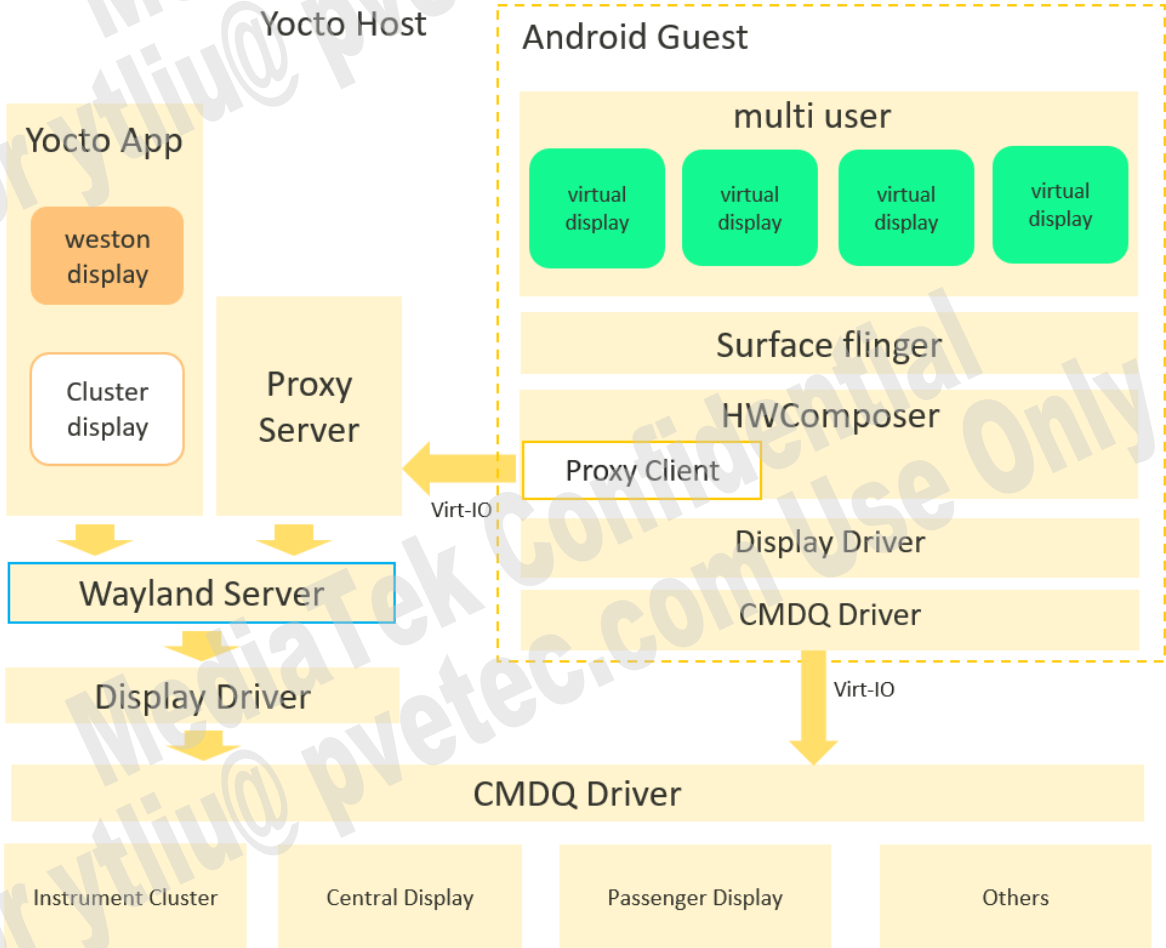
表 1-1. 缩略词

缩略词	全称	释义
API	Application Programming Interface	应用程序编程接口
HWC	Hardware Composer	硬件合成器，是 Android 系统中的一个组件，负责将应用程序界面与图形硬件进行交互，加速图形渲染和显示
RPC	Remote Procedure Call	远程过程调用
CMDQ	Command Queue	命令队列
DSI	Display Serial Interface	显示串行接口
eDP	Embedded DisplayPort	嵌入式显示端口
UI	User Interface	用户界面
IPO	Instant Power On	快速开机
OS	Operating System	操作系统

1.3 架构/进程概述

1.3.1 Hypervisor Proxy-Wayland 系统框架

Hypervisor Proxy-Wayland 系统框架如图 1-1 所示：



MediaTek Proprietary and Confidential. © 2022 MediaTek Inc. All rights reserved.

图 1-1. Hypervisor Proxy-Wayland 系统框架

1.3.2 Proxy-Wayland 模块结构分布

Proxy-Wayland 模块结构分布如图 1-2 所示：

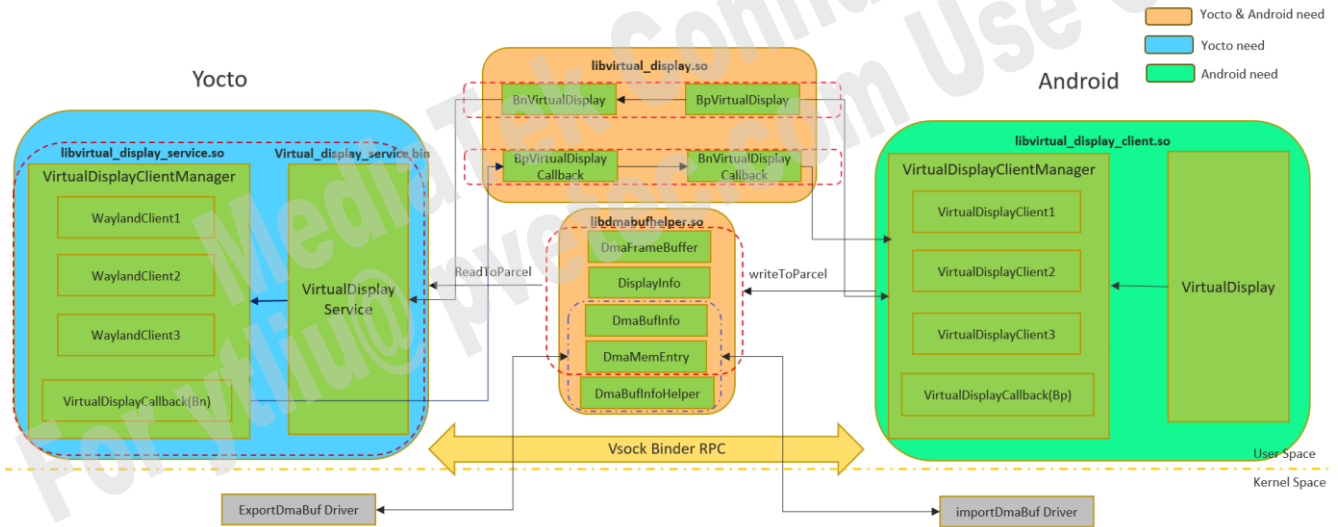


图 1-2. Proxy-Wayland 模块结构分布

1.4 测试与分析

1.4.1 Proxy-Wayland 功能性测试与分析

1.4.1.1 Proxy-Wayland Path 开机显示正常

• 测试

- 8668 Hyper Load + MT8668 Board
- 除了连接 DSI-0 屏，至少需要连接 DSI-1 屏和 DSI-2 屏
- 要求：开机后，待 Android 显示在 HOME UI，DSI-1 屏和 DSI-2 屏正常显示 Android 内容

• 常见问题分析

- 当出现无法显示的异常时，抓取 Yocto 端的 journal log，或设置 service file: lib/system/system/multi-virtual-display.service，在 StandardOutput 和 StandardError 行末分别加上 “+console”，并将 log level 设置为 6，保存并重启 Yocto device。通过 UART log 分析，有没有进行 Wayland client 的 connect 以及有没有 send frame 过来。
 - 若有 connect，但无 send frame，但 log 中出现 DROP FRAME，需要检查 log 中监测到的 display name 和 location 与 service file 中设置的 display 序数对应的 display 是否一致，若不一致，检查原因。

- 若有 send frame 但无显示, 需要进一步检查 log 的报错点, 按需与 proxy-wayland owner 一起检查问题发生的原因。
- 若没有 connect, 需要先抓 Android 端的 logcat, 检查 proxy client 有没有明显报错; 若没有 proxy client 相关 log, 即没有发现 send frame, 需要请 SF & HWC owner 检查相关原因。
- Yocto 端的 log 关键字 “PROXY_LOG”, Android 端的 log 关键字 “VirtualDisplay”
- 当出现一直卡着不刷图的异常时 (除静态 UI 外), 首先要确定不刷图的原因, 是 Android 端没有送图卡顿, 还是 Yocto 端主动 drop frame 导致。可以通过 Yocto 端的 log 来确认, 可以与 Proxy-Wayland owner 一起作第一手分析。
 - 若是 Android 端没有送图导致的卡住, 可以抓 logcat log 与 HWC & SF owner 一起分析。
 - 若是 proxy service 主动 drop frame, 可以同步抓 weston-debug log 与 weston owner 一起分析。

1.4.1.2 Proxy-Wayland 热插拔测试与分析

• 测试

- MT8668 Hyper Load + MT8668 Board
- 除了连接 DSI-0 屏, 至少需要连接 DSI-1 屏 和 DSI-2 屏
- 要求: 拔出 DSI-1 或 DSI-2, 对应屏的 Android 画面立即消失; 插上 DSI-1 或 DSI-2, 对应屏的 Android 画面恢复显示

• 常见问题分析

- 当拔出 DSI-1 或 DSI-2 时, 对应屏显示的 Android 画面没有消失, 显示在其它屏, 抓取对应的 journal 或 console log, 与 Proxy-Wayland owner 一起分析。
- 当重新插上 DSI-1 或 DSI-2 时, 对应屏显示的 Android 画面没有恢复显示, 抓取对应的 journal 或 console log, 与 Proxy-Wayland owner 一起分析。

1.4.2 Proxy-Wayland 稳定性测试与分析

1.4.2.1 帧率性能测试分析

• 测试数据打印准备

- 进入到 Yocto shell 修改 service file, `vi lib/systemd/system/multi-virtual-display.service`
- 在 “StandardOutput” 和 “StandardError” 行末 均加上 “+console”, 修改完保存, 然后重启 Yocto

- 测试中 UART log 中会出现 “D: x, C: y, render frame, fps: xxx”, 即 display x 的 client y render 1000 frame 的帧率是 xxx

• 测试方式 (以三屏为例)

- DSI-0 的中控显示 Android home UI
- DSI-1 或 DSI-2 开启 deskClock - Timer
- DSI-2 或 DSI-1 使用 gallery 播放 FHD@60 video

• 测试结果 (以三屏为例)

图 1-3 是 FHD@60 video 播放完, 两路 Proxy-Wayland display, 帧率数据的统计图:

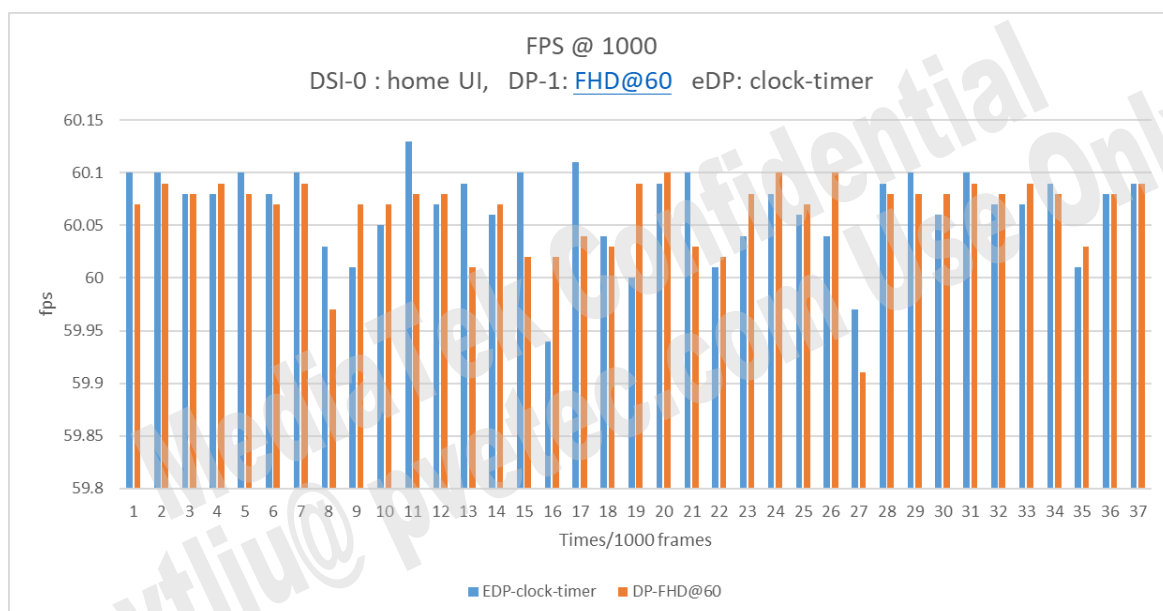


图 1-3. Proxy-Wayland two-path display fps

• 帧率问题分析

- 在检查多屏帧率中, 要先理清影响帧率的因素有哪些, 基于测试场景确定哪些是主要影响因素
- 根据模块区分, 影响因素有: GPU、Weston、Vsock 等。GPU 主要是表现在多屏多 user 使用 GPU 叠图导致的帧率下降; Weston 主要是表现在多屏中 Vsync 的处理带来的帧率下降; Vsock 主要表现在多 user 使用下导致的 Vsock 交互效率下降
- 通过设计对比实验确定主要影响因素, 主要是减少模块使用者个数, 看帧率是否得到改善
- 确定主要因素后, 抓 trace 分析请 module owner 分析影响帧率的原因

1.4.2.2 Weston Reboot 测试与分析

• 测试指令

- adb -s YOCTO shell "systemctl restart weston"

• 测试要求

- Weston restart 后， proxy-wayland path android 可以恢复正常显示， 帧率稳定后不受影响
- Weston restart 压测， 无相关 DB 生成

• Latency 测试

- Latency 计算范围
 - 起始时间： weston reboot， proxy server 的消息 loop 线程 wl_display_read_events failed
 - 终止时间： Proxy-Wayland client reconnect 成功后， 送去 wl_surface_commit 的第一帧
- Latency 计算方法
 - 在起始和终止两个点分别打印 date， 且输出在 journal 或 console log
 - 统计 10 次的差值， 并计算最大值、最小值和均值
- Latency 统计数据如图 1-4：

No.	wl_display_read_events failed Time	wl_surface_commit Time	latency time (ms)
1	147 ms @ Mon Sep 22:52:38 2024	631 ms @ Mon Sep 22:52:38 2024	484
2	140 ms @ Mon Sep 22:52:57 2024	699 ms @ Mon Sep 22:52:57 2024	559
3	711 ms @ Mon Sep 22:53:04 2024	299 ms @ Mon Sep 22:53:05 2024	588
4	368 ms @ Mon Sep 22:53:38 2024	819 ms @ Mon Sep 22:53:38 2024	451
5	389 ms @ Mon Sep 22:53:48 2024	959 ms @ Mon Sep 22:53:48 2024	570
6	240 ms @ Mon Sep 22:53:57 2024	840 ms @ Mon Sep 22:53:57 2024	600
7	546 ms @ Mon Sep 22:54:09 2024	146 ms @ Mon Sep 22:54:10 2024	600
8	536 ms @ Mon Sep 22:54:15 2024	993 ms @ Mon Sep 22:54:15 2024	457
9	829 ms @ Mon Sep 22:54:21 2024	284 ms @ Mon Sep 22:54:22 2024	455
10	398 ms @ Mon Sep 22:54:27 2024	896 ms @ Mon Sep 22:54:27 2024	498
		avg:	526.2
		max:	600
		min:	451

图 1-4. Weston reboot latency

• 测试问题可能性分析

- Weston reboot 后， Proxy-Wayland path Android 无法显示， 按 1.4.1.1 章节说到的无法正常显示问题处理
- Weston reboot 后， 帧率稳定后较之 reboot 前出现明显的下降。按帧率问题分析处理
- Weston reboot 压测， 生成相关 DB， 先解析 DB， 结合 log， 分析 DB 的原因， 确定问题发生点， 建议找 Proxy-Wayland owner 一起分析。

1.4.2.3 Proxy-Wayland Service Reboot 测试与分析

- 测试指令

- adb -s YOCTO shell "kill -9 \$(pgrep virtual_display)"

- 测试要求

- virtual_display_service reboot 后, proxy-wayland path android 可以恢复正常显示, 帧率稳定后不受影响
 - virtual_display_service reboot 压测, 无相关 DB 生成

- Latency 测试

- Latency 计算范围
 - 起始时间: 程序 signal handler 收到 sigkill 信号
 - 终止时间: 送去 wl_display_commit 的第一帧
 - Latency 计算方式
 - 在起始和终止两个点分别打印 date, 输出在 journal 或 console log
 - 统计 10 次的差值, 计算最大值、最小值和均值
 - Latency 统计数据, 由于 SIGKILL(9) 无法捕捉, 故采用 SIGTERM(15) 和 SIGINT(2) 两种方式统计
 - SIGTERM(15) 如图 1-5:

No. (15)	SIGTERM Hanlder	1st frame commit	Duration(ms)
1	52 ms @ Mon Sep 23 03:11:46 2024	363 ms @ Mon Sep 23 03:11:46 2024	311
2	684 ms @ Mon Sep 23 03:12:54 2024	840 ms @ Mon Sep 23 03:12:54 2024	156
3	285 ms @ Mon Sep 23 03:13:04 2024	605 ms @ Mon Sep 23 03:13:04 2024	320
4	659 ms @ Mon Sep 23 03:13:07 2024	855 ms @ Mon Sep 23 03:13:07 2024	196
5	474 ms @ Mon Sep 23 03:13:10 2024	854 ms @ Mon Sep 23 03:13:10 2024	380
6	776 ms @ Mon Sep 23 03:13:17 2024	112 ms @ Mon Sep 23 03:13:18 2024	336
7	339 ms @ Mon Sep 23 03:13:21 2024	613 ms @ Mon Sep 23 03:13:21 2024	274
8	815 ms @ Mon Sep 23 03:13:24 2024	110 ms @ Mon Sep 23 03:13:25 2024	295
9	92 ms @ Mon Sep 23 03:13:29 2024	361 ms @ Mon Sep 23 03:13:29 2024	269
10	197 ms @ Mon Sep 23 03:13:35 2024	610 ms @ Mon Sep 23 03:13:35 2024	413
		avg:	295
		max:	413
		min:	156

图 1-5. Proxy service reboot SIGTERM latency

- SIGINI(2) 如图 1-6:

No.(2)	SIGINT Hanlder	1st frame commit	Duration(ms)
1	707 ms @ Mon Sep 23 03:14:29 2024	102 ms @ Mon Sep 23 03:14:30 2024	395
2	140 ms @ Mon Sep 23 03:14:35 2024	349 ms @ Mon Sep 23 03:14:35 2024	209
3	526 ms @ Mon Sep 23 03:14:38 2024	847 ms @ Mon Sep 23 03:14:38 2024	321
4	346 ms @ Mon Sep 23 03:14:41 2024	614 ms @ Mon Sep 23 03:14:41 2024	268
5	167 ms @ Mon Sep 23 03:14:50 2024	345 ms @ Mon Sep 23 03:14:50 2024	178
6	886 ms @ Mon Sep 23 03:14:53 2024	111 ms @ Mon Sep 23 03:14:54 2024	225
7	644 ms @ Mon Sep 23 03:14:58 2024	860 ms @ Mon Sep 23 03:14:58 2024	216
8	613 ms @ Mon Sep 23 03:15:02 2024	859 ms @ Mon Sep 23 03:15:02 2024	246
9	135 ms @ Mon Sep 23 03:15:06 2024	353 ms @ Mon Sep 23 03:15:06 2024	218
10	671 ms @ Mon Sep 23 03:15:12 2024	853 ms @ Mon Sep 23 03:15:12 2024	182
		avg:	245.8
		max:	395
		min:	178

图1-6. Proxy service reboot SIGINT latency

• 测试问题可能性分析

- 无法恢复正常显示，按无法正常显示问题分析
- 帧率稳定后与 reboot 前有明显下降，按帧率问题处理
- 压测有相关 DB 生成，与 Proxy-Wayland owner 一起分析问题发生点

1.4.2.4 Android Reboot 测试与分析

• 测试指令

- `adb -s 0123456789ABCDEF shell "reboot"`

• 测试要求

- Android reboot 后，proxy-wayland path android 可以恢复正常显示
- Android reboot 压测，无相关 DB 生成

• Latency 测试

- Latency 计算范围
 - 起始时间：由于 Android reboot，proxy service binderDied 被调用
 - 终止时间：送去 wl_surface_commit 的第一帧
- Latency 计算方式
 - 在起始和终止两个点分别打印 date，输出在 journal 或 console log
 - 统计 10 次的差值，计算最大值、最小值和均值
- Latency 统计数据，下面是一份在 userdebug load 统计的数据，仅当参考：

No.	binderDied	1st Frame Commit	duration(s.ms)	ms
1	904 ms @ Mon Sep 23 02:25:10 2024	346 ms @ Mon Sep 23 02:25:51 2024	40s 442ms	40442
2	625 ms @ Mon Sep 23 02:26:16 2024	664 ms @ Mon Sep 23 02:26:56 2024	40s 39ms	40039
3	720 ms @ Mon Sep 23 02:27:09 2024	487 ms @ Mon Sep 23 02:27:49 2024	39s 767ms	39767
4	340 ms @ Mon Sep 23 02:28:51 2024	943 ms @ Mon Sep 23 02:29:30 2024	39s 603ms	39603
5	223 ms @ Mon Sep 23 02:32:21 2024	606 ms @ Mon Sep 23 02:33:00 2024	39s 383ms	39383
6	296 ms @ Mon Sep 23 02:33:14 2024	271 ms @ Mon Sep 23 02:33:54 2024	39s 975ms	39975
7	838 ms @ Mon Sep 23 02:34:23 2024	862 ms @ Mon Sep 23 02:35:04 2024	41s 24ms	41024
8	429 ms @ Mon Sep 23 02:35:14 2024	933 ms @ Mon Sep 23 02:35:53 2024	39s 504ms	39504
9	172 ms @ Mon Sep 23 02:36:01 2024	403 ms @ Mon Sep 23 02:36:40 2024	39s 231ms	39231
10	390 ms @ Mon Sep 23 02:36:50 2024	354 ms @ Mon Sep 23 02:37:30 2024	39s 964ms	39964
		avg:	39s 893ms	39893.2
		max:	41s 24ms	41024
		min:	39s 231ms	39231

图 1-7. Android reboot latency

测试问题可能性分析

- Android reboot 后，无法恢复正常显示，按无法显示问题处理
- Android reboot 压测，有 DB 生成，与 Proxy-Wayland owner 一起第一手分析

1.4.2.5 Android IPO 指令测试与分析

测试指令

- adb -s 0123456789ABCDEF shell service call power 67
- adb -s 0123456789ABCDEF shell service call power 68 i32 1

测试要求

- 执行第一句指令，proxy-wayland path android 画面消失；执行第二句指令，proxy-wayland path android 画面恢复正常显示。
- Android IPO 指令压测，无相关 DB 生成

测试问题可能性分析

- 执行第一句测试指令，proxy android 画面无法退出，抓取 logcat log 检查有没有执行 proxy client disconnect；若有执行，结合 proxy server log 分析 disconnect flow 是否正常执行
- 执行第二句测试指令，proxy android 画面无法显示，按无法显示异常问题处理
- Android IPO 压测出现异常且有 DB 生成，与 Proxy-Wayland owner 一起作第一手分析

1.4.2.6 Android 灭屏与亮屏测试与分析

测试指令

- adb -s 0123456789ABCDEF shell input keyevent 26, 执行两次

- **测试要求**

- 执行灭屏时, proxy android 画面消失; 执行亮屏时, proxy android 画面恢复正常显示
- 灭屏与亮屏的压测, 行为正常, 且无 DB 生成

- **测试问题可能性分析**

- 若执行灭屏时, proxy android 画面无法消失, 抓取 logcat log 检查有没有执行 proxy client 的 disconnect, 若有执行, 结合 proxy server log 分析 disconnect flow 是否正常执行
- 若执行亮屏时, proxy android 画面无法恢复显示, 按无法显示异常问题处理
- 灭屏与亮屏压测出现异常且有 DB 生成, 与 Proxy-Wayland owner 一起分析

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。